APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU CST414 — DEEP LEARNING

MODULE 2 HANDWRITTEN LECTURE NOTES

B.Tech Computer Science and Engineering · Scheme 2019

Course Code: CST414	Credits: 3
Course Name: Deep Learning	Lecture Hours: 36
Faculty In-Charge: Jane Rose	Department: Computer Science & Engineering
Design: High-Contrast Printable Notebook	Font: Authentic Chalk & Kalam Ink

Prepared for students and faculty for classroom lectures, print study, and university examination preparation.

COURSE LECTURE INDEX & MAPPING

Day	Topic ID	Lecture Title	Module	CO	Date
#08	2.1	Introduction to Deep Learning and Deep F...	M2	CO2	12-Sep-2025
#09	2.2	Training Deep Models: Optimization and R...	M2	CO2	15-Sep-2025
#10	2.3	Gradient Descent (GD) and GD with Moment...	M2	CO2	17-Sep-2025
#11	2.4	Nesterov Accelerated Gradient (NAG) and ...	M2	CO2	19-Sep-2025
#12	2.5	Adaptive Learning Rate Methods: AdaGrad,...	M2	CO2	22-Sep-2025
#13	2.6	Regularization Techniques: L1, L2, Early...	M2	CO2	24-Sep-2025
#14	2.7	Parameter Sharing, Parameter Tying, and ...	M2	CO2	26-Sep-2025
#15	2.8	Ensemble Methods, Model Averaging, Baggi...	M2	CO2	29-Sep-2025
#16	2.9	Dropout and Parameter Initialization	M2	CO2	03-Oct-2025

LECTURE #08 · MODULE 2 (Topic 2.1)

Date: 12-Sep-2025
Period: Period 1

Title: Introduction to Deep Learning and Deep Feed-Forward Networks

CO: CO2 Bloom's: Understand

Prerequisites: Multi-Layer Perceptrons, Linear Algebra, Machine Learning fundamentals
Key Terminology: Deep Learning, Deep Feedforward Network (DFN), Hierarchical Feature Extraction, Circuit Complexity, Representation Depth, Inductive Bias

Introduction

Deep learning is a subfield of machine learning based on artificial neural networks with representation learning. The adjective 'deep' refers to the use of multiple layers in the network. A deep feedforward network (DFN), also called a multilayer perceptron, defines a mapping y = f(x; theta) and learns the value of the parameters theta that result in the best function approximation.

Depth Vs Width

Theoretical justification for depth: 1. Exponential Efficiency: Certain functions requiring an exponential number of hidden units in a single-layer network can be approximated to arbitrary accuracy using a deep network with a polynomial number of units (Montufar et al., 2014; Telgarsky, 2016). 2. Compositional Hierarchy: Real-world perceptual data (vision, audio, natural language) is inherently compositional. Deep networks mirror this physics: early layers extract low-level primitives (edges, frequencies, phonemes), middle layers compose parts (textures, contours, syllables), and deep layers represent high-level semantic concepts (faces, words, objects). 3. Inductive Bias: Depth enforces the assumption that the target function is a composition of simpler functions, greatly constraining the search space and improving sample efficiency.

Mathematical Composition

*A deep network with L hidden layers defines a chain of function compositions: f(x) = f^(L)(f^(L-1)(... f^(2)(f^(1)(x)) ...)) where each layer computes: h^(l) = g^(l)((W^(l))^T * h^(l-1) + b^(l)), with h^(0) = x. The parameters are the collective set theta = { W^(1), b^(1), ..., W^(L), b^(L) }.*

Exam Important Points

- *Contrast shallow feature engineering with end-to-end deep representation learning.*
- *Why deep architectures are exponentially more parameter-efficient than shallow ones.*
- *Function composition formulation of Deep Feed-forward Networks.*
- *The three primary driving forces behind the deep learning revolution: Big Data, GPU parallelism, Algorithmic breakthroughs (ReLU, Adam, Dropout).*

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Layer Dimensions and Memory Footprint in Deep Feed-Forward Network

Statement: A deep feedforward network has architecture [100, 50, 25, 10, 2]. Each parameter and activation is stored as a 32-bit floating-point number (4 bytes). Calculate: (a) Total number of weight parameters, (b) Total number of bias parameters, (c) Parameter memory consumption in kilobytes, and (d) Activation memory required to store the forward pass of a mini-batch of 64 samples.

Given: Dims: [100, 50, 25, 10, 2], float32 (4 bytes/val), batch size B = 64.

Governing Formula: W_total = sum(n_l * n_{l-1}); B_total = sum(n_l); Params_mem = (W_total + B_total)*4 / 1024; Act_mem = B * sum(n_l)*4 / 1024

Step-by-Step Calculation:

Step 1: Weights calculation: W1 = 50 * 100 = 5,000 W2 = 25 * 50 = 1,250 W3 = 10 * 25 = 250 W4 = 2 * 10 = 20 Total Weights = 5,000 + 1,250 + 250 + 20 = 6,520.

Step 2: Biases calculation: B1 = 50, B2 = 25, B3 = 10, B4 = 2 Total Biases = 50 + 25 + 10 + 2 = 87.

Step 3: Total Parameters = 6,520 + 87 = 6,607 parameters. Parameter Memory = 6,607 * 4 bytes = 26,428 bytes = 25.81 KB.

Step 4: Activations per sample (all layers 1 to 4) = 50 + 25 + 10 + 2 = 87 activations. For batch of 64: Total activation floats = 64 * 87 = 5,568 floats. Activation Memory = 5,568 * 4 bytes = 22,272 bytes = 21.75 KB.

[FINAL EVALUATED ANSWER: Weights = 6,520, Biases = 87, Total Params = 6,607 (25.81 KB), Batch Activation Memory = 21.75 KB]

Pedagogical Note: In deep feedforward networks, parameter storage is modest for dense layers of moderate width, but activation caching scales linearly with mini-batch size.

Problem 2 (Advanced (KTU Exam Level)): Theoretical Piecewise Linear Regions Bound in Deep vs Shallow ReLU Networks

Statement: According to Montufar et al. (2014), a deep neural network with L layers of width n utilizing ReLU activations can partition the input space into Omega((n/n_in)^(n_in * (L-1)) * n^n_in) linear regions, whereas a shallow 1-hidden-layer network with total neurons N can partition the space into at most sum_{j=0}^{n_in} binom(N, j) regions. For an input dimension n_in = 2: (a) Calculate the upper bound of linear regions for a shallow network with N = 40 neurons, and (b) Compute the exponential lower bound for a deep network having L = 4 hidden layers of width n = 10 each (same total neurons N = 40).

Given: n_in = 2, Shallow: N = 40; Deep: 4 layers with n = 10 each (total 40 neurons).

Governing Formula: Shallow bound = binom(N, 0) + binom(N, 1) + binom(N, 2) = 1 + N + N*(N-1)/2; Deep bound approx (n/2)^(2*(L-1)) * n^2 = (10/2)^6 * 10^2 = 5^6 * 100

Step-by-Step Calculation:

Step 1: Compute shallow network linear regions: Regions_shallow = 1 + 40 + (40 * 39 / 2) = 1 + 40 + 780 = 821 regions.

Step 2: Compute deep network linear regions: With L=4 layers of width 10, the composition of folding piecewise linear operations scales as (n/2)^(d*(L-1)) * n^d. For d = 2, (10/2)^(2 * (4 - 1)) * 10^2 = 5^(6) * 100 = 15,625 * 100 = 1,562,500 regions!

[FINAL EVALUATED ANSWER: Shallow network (40 neurons) = 821 regions. Deep network (4x10 = 40 neurons) = ~1,562,500 regions.]

Pedagogical Note: With the exact same total budget of 40 hidden neurons, distributing them across 4 sequential layers creates over 1,900 times more expressive piecewise linear decision boundaries than arranging them in a single shallow layer.

Real-World Industry Case (Smart Industry 4.0 & Industrial Internet of Things (IIoT)): Industrial High-Dimensional Predictive Maintenance

Pipeline Workflow: High-frequency vibration, thermal, and acoustic sensor arrays on wind turbines -> 6-layer Deep Feedforward Network extracts non-linear failure precursors -> Predicts remaining useful life (RUL) in hours.

Industrial Impact & Benefit: Enables proactive component replacement prior to catastrophic gearbox seizure, saving over $250,000 per turbine incident.

Key Oral Viva Voce Checkpoints

Q1: What distinguishes deep learning from classical machine learning?

Ans: Deep learning automatically extracts hierarchical feature representations directly from raw data without manual feature engineering.

Q2: Why is a deep feedforward network called 'feedforward'?

Ans: Because information flows strictly forward from input nodes through hidden units to output nodes without feedback connections or recurrence.

Q3: What is the theoretical advantage of depth over width?

Ans: Deep architectures can compute complex functions with exponentially fewer parameters by reusing sub-features computed in earlier layers.

Q4: What is an inductive bias in neural networks?

Ans: The set of architectural assumptions the model uses to predict outputs on unseen inputs, such as hierarchical compositionality in deep networks.

Q5: Why did deep networks fail to train effectively prior to 2006-2010?

Ans: Lack of massive labeled datasets, lack of GPU compute, poor weight initialization, and vanishing gradients caused by sigmoid/tanh activations.

LECTURE #09 · MODULE 2 (Topic 2.2)

Date: 15-Sep-2025
Period: Period 2

Title: Training Deep Models: Optimization and Regularization Overview

CO: CO2 Bloom's: Understand

Prerequisites: Loss Functions, Gradient Calculus, Day 8 concepts
Key Terminology: Optimization vs Generalization, Surrogate Loss Function, Regularization, Generalization Error, Double Descent, Bias-Variance Tradeoff

Optimization Vs Machine Learning

In traditional numerical optimization, the objective is to minimize a known function J(theta). In machine learning, our true goal is to minimize the expected risk over the unseen data-generating distribution p_data(x, y): J*(theta) = E_{(x,y)~p_data} [L(f(x; theta), y)] We can only compute the empirical risk on training data. Moreover, machine learning often optimizes a surrogate loss function (e.g. cross-entropy loss) because the true metric (0-1 classification error) is non-smooth, non-differentiable, and NP-hard to optimize.

Regularization Definition

Ian Goodfellow defines regularization as: 'Any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error.' Regularization Taxonomy in Deep Learning: 1. Parameter Penalties: Adding norm constraints to the objective (L1, L2 weight decay). 2. Explicit Constraints: Hard parameter projection, weight clipping. 3. Dataset Modification: Data augmentation, input noise injection. 4. Architectural Priors: Convolutional parameter sharing, pooling. 5. Stochastic Noise: Dropout, DropConnect, label smoothing. 6. Algorithmic Dynamics: Early stopping, learning rate scheduling.

Bias Variance And Double Descent

Classical Regime: As model capacity increases, bias decreases monotonically while variance increases, producing a U-shaped test error curve. Modern Interpolation Regime (Belkin et al., 2019): Beyond the interpolation threshold (where the network perfectly memorizes training data with zero training error), test error undergoes a 'Double Descent' and decreases again! High overparameterization combined with stochastic gradient descent implicitly selects minimum-norm interpolating solutions that generalize remarkably well.

Exam Important Points

- Formal definition of Regularization in Deep Learning.
- Difference between pure optimization and machine learning optimization.
- Why surrogate loss functions are necessary.
- Sketch and explanation of classical bias-variance tradeoff vs modern double descent curve.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Empirical Risk vs Surrogate Loss Evaluation

Statement: A binary classifier evaluates 4 test samples with true labels y in {0, 1}. The predicted probabilities p = P(y=1) are [0.90, 0.40, 0.20, 0.85] for true labels [1, 1, 0, 0]. Using a decision threshold of 0.5: (a) Calculate the 0-1 classification error (true empirical risk), and (b) Calculate the Binary Cross-Entropy surrogate loss.

Given: y = [1, 1, 0, 0], p = [0.90, 0.40, 0.20, 0.85], threshold = 0.5.

Governing Formula: 0-1 error: I(y_pred != y); L_BCE = - (1/N) * sum [y_i * ln(p_i) + (1 - y_i) * ln(1 - p_i)]

Step-by-Step Calculation:

Step 1: Compute threshold predictions: p1=0.90 -> 1 (Correct, error=0) p2=0.40 -> 0 (Incorrect, error=1) p3=0.20 -> 0 (Correct, error=0) p4=0.85 -> 1 (Incorrect, error=1). Total misclassifications = 2 out of 4.

Step 2: Calculate 0-1 error: Error = 2 / 4 = 0.50 (50%).

Step 3: Calculate individual cross-entropy losses: L1 = - ln(0.90) = - (-0.1054) = 0.1054 L2 = - ln(0.40) = - (-0.9163) = 0.9163 L3 = - ln(1 - 0.20) = - ln(0.80) = - (-0.2231) = 0.2231 L4 = - ln(1 - 0.85) = - ln(0.15) = - (-1.8971) = 1.8971.

Step 4: Average BCE loss: L_avg = (0.1054 + 0.9163 + 0.2231 + 1.8971) / 4 = 3.1419 / 4 = 0.7855.

[FINAL EVALUATED ANSWER: 0-1 Classification Error = 0.50 (50%), Cross-Entropy Surrogate Loss = 0.7855]

Pedagogical Note: Sample 4 contributed the largest penalty (1.8971) because the model was highly confident (85%) in the wrong direction, demonstrating why surrogate log loss provides a smooth, informative gradient for training.

Problem 2 (Advanced (KTU Exam Level)): Bias-Variance Decomposition of Mean Squared Error

Statement: The expected generalization error of a machine learning estimator f_hat(x) can be decomposed as E[(y - f_hat(x))^2] = Bias^2[f_hat(x)] + Var[f_hat(x)] + sigma_epsilon^2, where the true target is y = f(x) + epsilon with noise variance sigma_epsilon^2 = 0.05. An unregularized deep network has Bias = 0.10 and Variance = 0.40. An appropriately regularized network reduces Variance to 0.12 while slightly increasing Bias to 0.15. Calculate the total expected test MSE for both models and determine the percentage reduction achieved through regularization.

Given: sigma_epsilon^2 = 0.05. Model 1 (Unreg): Bias = 0.10, Var = 0.40. Model 2 (Reg): Bias = 0.15, Var = 0.12.

Governing Formula: MSE = Bias^2 + Variance + Irreducible_Noise; Reduction = (MSE1 - MSE2) / MSE1 * 100

Step-by-Step Calculation:

Step 1: Compute MSE for Unregularized Model 1: Bias^2 = (0.10)^2 = 0.010. Var = 0.40. Noise = 0.05. MSE_1 = 0.010 + 0.40 + 0.05 = 0.460.

Step 2: Compute MSE for Regularized Model 2: Bias^2 = (0.15)^2 = 0.0225. Var = 0.12. Noise = 0.05. MSE_2 = 0.0225 + 0.12 + 0.05 = 0.1925.

Step 3: Calculate percentage error reduction: Delta = 0.460 - 0.1925 = 0.2675. Percentage Reduction = (0.2675 / 0.460) * 100 = 58.15%.

[FINAL EVALUATED ANSWER: Unregularized MSE = 0.460, Regularized MSE = 0.1925 (58.15% error reduction)]

Pedagogical Note: Although regularization slightly increased the model's squared bias (+0.0125), it slashed variance by 0.28, cutting overall generalization error by over 58%.

Real-World Industry Case (Quantitative Finance & Algorithmic Execution): Real-Time Stock Market Algorithmic Trading

Pipeline Workflow: High-frequency limit order book feeds tick-level pricing into deep network -> Pure unregularized optimization overfits to market microstructure noise -> Multi-tier regularization (L2 penalty + early stopping) ensures model generalizes across volatile macroeconomic shifts.

Industrial Impact & Benefit: Prevents catastrophic portfolio drawdown during unexpected market volatility spikes.

Key Oral Viva Voce Checkpoints

Q1: What is Goodfellow's formal definition of regularization?

Ans: Any modification made to a learning algorithm intended to reduce its generalization error but not its training error.

Q2: What is a surrogate loss function?

Ans: A smooth, differentiable proxy loss (such as cross-entropy) optimized in place of the intractable true objective (such as 0-1 misclassification error).

Q3: Why is 0-1 loss intractable for gradient-based training?

Ans: Its derivative is zero almost everywhere and undefined at the decision boundary, offering no gradient direction for parameter updates.

Q4: Explain the concept of 'Double Descent'.

Ans: The observation that as model capacity increases past the interpolation threshold (where training error reaches zero), test error decreases again rather than continually climbing.

Q5: What is the difference between empirical risk and true risk?

Ans: Empirical risk is the average loss on the observed training sample; true risk is the expected loss over the entire underlying data distribution.

LECTURE #10 · MODULE 2 (Topic 2.3)

Date: 17-Sep-2025
Period: Period 4

Title: Gradient Descent (GD) and GD with Momentum

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Vector Calculus (gradient vector), First-order Taylor Series, Day 7 concepts
Key Terminology: Batch Gradient Descent, Momentum, Velocity Vector, Friction Coefficient (Beta), Exponentially Weighted Moving Average, Ravine Oscillation

Gradient Descent

Standard Gradient Descent (Cauchy, 1847) updates parameters theta in the direction of the negative gradient: theta_{t+1} = theta_t - eta * grad_theta J(theta_t) Failure Modes: 1. Ravines (Ill-conditioned curvature): The gradient points across the steep valley rather than along the gentle slope toward the minimum, causing severe zig-zagging. 2. Saddle Points & Plateaus: Near critical points, ||grad J|| -> 0, causing the parameter updates to stall. 3. Local Minima: Without kinetic energy, GD is permanently trapped in any local depression.

Momentum Algorithm

Polyak (1964) introduced the heavy-ball / momentum method, simulating a physical particle with mass moving down a potential energy surface subject to friction. Mathematical Formulation: v_{t+1} = beta * v_t + (1 - beta) * grad_theta J(theta_t) [or simply: v_{t+1} = beta * v_t - eta * grad J] theta_{t+1} = theta_t + v_{t+1} (where beta in [0, 1) is the momentum coefficient, commonly set to 0.9). Mechanisms of Acceleration: - In directions where gradients alternate signs (oscillating valley walls), consecutive updates cancel out, damping oscillations. - In directions where gradients consistently point in the same direction, velocity accumulates up to a terminal factor of 1 / (1 - beta) times the base step size! - For beta = 0.9, the effective step size along consistent slopes is amplified tenfold: 1 / (1 - 0.9) = 10x.

Exam Important Points

- Full equations for Vanilla GD vs GD with Momentum.
- Physical interpretation of the momentum parameter beta as friction/inertia.
- Mathematical derivation of the terminal velocity factor 1 / (1 - beta).
- Graphical comparison of trajectories on an ill-conditioned quadratic bowl.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Two-Step Momentum Velocity and Weight Calculation

Statement: An optimizer with momentum coefficient beta = 0.9 and learning rate eta = 0.1 starts with initial parameter w_0 = 5.0 and initial velocity v_0 = 0.0. The gradients observed at step 0 and step 1 are g_0 = 2.0 and g_1 = 1.8. Using the update rule v_{t+1} = beta * v_t - eta * g_t, w_{t+1} = w_t + v_{t+1}, calculate the exact velocity and parameter values at steps t = 1 and t = 2.

Given: w_0 = 5.0, v_0 = 0.0, beta = 0.9, eta = 0.1, g_0 = 2.0, g_1 = 1.8
Governing Formula: v_{t+1} = beta * v_t - eta * g_t; w_{t+1} = w_t + v_{t+1}

Step-by-Step Calculation:

Step 1 (t = 1): v_1 = (0.9 * 0.0) - (0.1 * 2.0) = 0.0 - 0.20 = -0.20. w_1 = 5.0 + (-0.20) = 4.80.
Step 2 (t = 2): v_2 = (0.9 * -0.20) - (0.1 * 1.8) = -0.18 - 0.18 = -0.36. w_2 = 4.80 + (-0.36) = 4.44.

[FINAL EVALUATED ANSWER: At step 1: v_1 = -0.20, w_1 = 4.80. At step 2: v_2 = -0.36, w_2 = 4.44.]
Pedagogical Note: Even though the gradient decreased from 2.0 to 1.8, the accumulated velocity increased from -0.20 to -0.36, accelerating the parameter step toward the minimum.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q13(b): GD with vs without Momentum on Plateau

Statement: Differentiate gradient descent with and without momentum. Consider a flat plateau where the gradient is constant at g = 0.05. The learning rate is eta = 0.02 and momentum factor beta = 0.9. (a) Calculate the steady-state parameter step size for vanilla GD, (b) Derive the terminal velocity step size for GD with momentum, and (c) Determine how many times faster momentum traverses the plateau compared to vanilla GD.

Given: Constant gradient g = 0.05, eta = 0.02, beta = 0.9.

Governing Formula: Vanilla step = - eta * g; Terminal velocity v_inf = - (eta * g) / (1 - beta); Speedup = v_inf / (eta * g) = 1 / (1 - beta)

Step-by-Step Calculation:

Step 1: Compute Vanilla GD step size: step_gd = eta * g = 0.02 * 0.05 = 0.0010.

Step 2: Derive Momentum recursive sequence under constant gradient g: v_1 = - eta * g v_2 = beta * v_1 - eta * g = - eta * g * (1 + beta) v_t = - eta * g * sum_{k=0}^{t-1} beta^k. As t -> infinity, sum_{k=0}^infinity beta^k = 1 / (1 - beta).

Step 3: Calculate terminal velocity: v_inf = - (0.02 * 0.05) / (1 - 0.9) = - 0.0010 / 0.10 = - 0.0100.

Step 4: Compute acceleration ratio: Ratio = |v_inf| / step_gd = 0.0100 / 0.0010 = 10.0x.

[FINAL EVALUATED ANSWER: Vanilla step = 0.0010, Momentum terminal step = 0.0100. Speedup = 10x faster.]

Pedagogical Note: On flat plateaus or persistent gradients, momentum accelerates progress tenfold by accumulating velocity over past steps.

Real-World Industry Case (Robotics & Real-Time Motion Control): Robotic Arm Inverse Kinematics Trajectory Planning

Pipeline Workflow: 6-DOF robotic manipulator computes joint angles for target end-effector coordinates -> High non-linearity creates flat plateaus and steep singularities in joint space -> Momentum-accelerated optimization prevents joint velocity stalling.

Industrial Impact & Benefit: Reduces robotic pick-and-place cycle times on automotive assembly lines by 24%.

Key Oral Viva Voce Checkpoints

Q1: What is the physical analogy behind gradient descent with momentum?

Ans: A heavy ball rolling down a potential energy surface, accumulating kinetic momentum while subject to viscous friction.

Q2: What is the typical value used for the momentum hyperparameter beta?

Ans: 0.9 (and occasionally 0.99 for deep architectures).

Q3: How does momentum resolve the problem of oscillating gradients in narrow ravines?

Ans: Oscillating components across the valley walls have opposing signs and cancel each other out over successive steps, while consistent components along the ravine floor reinforce each other.

Q4: What is the terminal velocity amplification factor of momentum?

Ans: 1 / (1 - beta); for beta = 0.9, the effective step size is amplified by a factor of 10.

Q5: Can momentum cause the optimizer to overshoot the global minimum?

Ans: Yes, if the accumulated velocity is too high, it may overshoot the minimum and oscillate around it before settling.

LECTURE #11 · MODULE 2 (Topic 2.4)

Date: 19-Sep-2025
Period: Period 1

Title: Nesterov Accelerated Gradient (NAG) and Stochastic Gradient Descent (SGD)

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Gradient Descent, Momentum Algorithm, Day 10 concepts
Key Terminology: Nesterov Accelerated Gradient, Look-Ahead Step, Stochastic Gradient Descent, Mini-Batch SGD, Robbins-Monro Conditions, Gradient Noise

Nesterov Accelerated Gradient

Yurii Nesterov (1983) modified classical momentum by computing the gradient not at the current position theta_t, but at the 'look-ahead' position (theta_t + beta * v_t): NAG Equations: 1. Look-ahead projection: theta_lookahead = theta_t + beta * v_t 2. Evaluate gradient at projected point: g_t = grad_theta J(theta_lookahead) 3. Update velocity: v_{t+1} = beta * v_t - eta * g_t 4. Update parameters: theta_{t+1} = theta_t + v_{t+1} Advantage: NAG acts as an adaptive brake. If the current velocity is carrying the parameters uphill, the look-ahead gradient detects the slope increase BEFORE the step is taken and immediately applies a counter-acting brake, significantly reducing overshooting.

Stochastic Gradient Descent

Standard Batch GD computes the exact gradient over all N training examples: cost O(N) per step. For millions of samples, this is computationally prohibitive. Stochastic Gradient Descent (Robbins & Monro, 1951) approximates the true gradient using a single randomly chosen sample (or a mini-batch B of size 32 to 512): g_t = (1 / |B|) * sum_{i in B} grad_theta L(f(x_i; theta), y_i) theta_{t+1} = theta_t - eta_t * g_t Robbins-Monro Convergence Conditions: To guarantee almost-sure convergence to a local minimum under stochastic noise: 1. sum_{t=1}^infinity eta_t = infinity (learning rate must allow traversing any distance) 2. sum_{t=1}^infinity eta_t^2 < infinity (learning rate must shrink fast enough to damp noise)

Convergence Comparison

Theoretical convergence rates on convex L-smooth functions: - Standard Gradient Descent: O(1 / k) after k iterations. - Nesterov Accelerated Gradient: O(1 / k^2) — proved to be the optimal theoretical lower bound for first-order black-box optimization!

Exam Important Points

- Formulate NAG update equations and contrast with classical Polyak momentum.
- Diagram illustrating the look-ahead step in Nesterov momentum.
- Formulation of Mini-Batch SGD and its computational trade-offs.
- Robbins-Monro conditions for convergence of stochastic gradient descent.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Nesterov Look-Ahead Position and Gradient Evaluation

Statement: An optimizer using Nesterov momentum has current parameter w_t = 2.0, current velocity v_t = -0.5, and momentum factor beta = 0.9. The loss function is J(w) = 0.5 * w^2. Learning rate is eta = 0.1. (a) Calculate the look-ahead position w_ahead, (b) Compute the gradient at the look-ahead position, and (c) Compute the updated velocity v_{t+1} and new parameter w_{t+1}.

Given: w_t = 2.0, v_t = -0.5, beta = 0.9, J(w) = 0.5 * w^2, eta = 0.1.

Governing Formula: w_ahead = w_t + beta * v_t; g_t = dJ/dw(w_ahead) = w_ahead; v_{t+1} = beta * v_t - eta * g_t; w_{t+1} = w_t + v_{t+1}

Step-by-Step Calculation:

Step 1: Compute look-ahead position: w_ahead = 2.0 + (0.9 * -0.5) = 2.0 - 0.45 = 1.55.
Step 2: Evaluate gradient at look-ahead position: g_ahead = dJ/dw(1.55) = 1.55.
Step 3: Update velocity: v_{t+1} = (0.9 * -0.5) - (0.1 * 1.55) = -0.45 - 0.155 = -0.605.
Step 4: Update parameter: w_{t+1} = 2.0 + (-0.605) = 1.395.

[FINAL EVALUATED ANSWER: w_ahead = 1.55, Gradient = 1.55, v_{t+1} = -0.605, w_{t+1} = 1.395]

Pedagogical Note: Nesterov evaluated the slope where momentum was about to carry the parameter (at 1.55) rather than at the stale position (2.0), adjusting velocity dynamically.

Problem 2 (Advanced (KTU Exam Level)): Stochastic Gradient Variance Reduction with Mini-Batch Size

Statement: Let the true population gradient be g_true with variance Var(g_single) = sigma^2 = 16.0 when computed on a single random sample. (a) Calculate the variance of the mini-batch gradient estimator for batch sizes B = 1, B = 16, B = 64, and B = 256, (b) Determine the standard error of the gradient estimate for each batch size, and (c) Discuss the computational trade-off between variance reduction and matrix multiply saturation on a GPU.

Given: sigma^2 = 16.0 for B = 1.

Governing Formula: Var(g_B) = sigma^2 / B; SE(g_B) = sqrt(Var(g_B)) = sigma / sqrt(B)

Step-by-Step Calculation:

Step 1: For B = 1: Var = 16.0 / 1 = 16.0; SE = sqrt(16.0) = 4.00.
Step 2: For B = 16: Var = 16.0 / 16 = 1.0; SE = sqrt(1.0) = 1.00.
Step 3: For B = 64: Var = 16.0 / 64 = 0.25; SE = sqrt(0.25) = 0.50.
Step 4: For B = 256: Var = 16.0 / 256 = 0.0625; SE = sqrt(0.0625) = 0.25.

[FINAL EVALUATED ANSWER: B=1: SE=4.00; B=16: SE=1.00; B=64: SE=0.50; B=256: SE=0.25.]

Pedagogical Note: Increasing batch size from 1 to 64 reduces gradient standard error by a factor of 8 (from 4.0 to 0.5). Beyond B=256, returns diminish as diminishing variance gains require quadrupling the batch size to halve error, while GPU memory fills.

Real-World Industry Case (Edge AI & Drone Surveillance Systems): Real-Time Embedded Video Object Tracking

Pipeline Workflow: Drone onboard camera streams 60 fps 4K video -> High frame rate requires sub-15ms model updates on edge GPU -> Mini-Batch SGD with Nesterov momentum optimizes lightweight detector parameters on-the-fly without tracking lag.

Industrial Impact & Benefit: Enables uninterrupted drone camera lock-on for search-and-rescue target tracking in mountain terrain.

Key Oral Viva Voce Checkpoints

Q1: What is the key difference between Polyak's standard momentum and Nesterov momentum?

Ans: Polyak evaluates the gradient at the current position theta_t, whereas Nesterov evaluates the gradient at the look-ahead position (theta_t + beta*v_t).

Q2: What is the convergence rate of Nesterov Accelerated Gradient for convex smooth functions?

Ans: O(1 / k^2), which is significantly faster than standard GD's O(1 / k).

Q3: Why is pure SGD (batch size = 1) computationally inefficient on GPUs?

Ans: GPUs achieve high compute utilization through massive vector and matrix parallelization; a single sample fails to saturate thousands of CUDA cores.

Q4: State the Robbins-Monro conditions for stochastic optimization.

Ans: The learning rate sequence eta_t must satisfy: sum_{t=1}^infty eta_t = infty and sum_{t=1}^infty eta_t^2 < infty.

Q5: Why is the first Robbins-Monro condition necessary?

Ans: To ensure the optimizer has sufficient energy to travel an arbitrarily large distance in parameter space to reach the minimum regardless of initialization.

LECTURE #12 · MODULE 2 (Topic 2.5)

Date: 22-Sep-2025
Period: Period 2

Title: Adaptive Learning Rate Methods: AdaGrad, RMSProp, and Adam

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Gradient Descent with Momentum, Element-wise vector operations, Day 11 concepts
Key Terminology: AdaGrad, RMSProp, Adam (Adaptive Moment Estimation), Second Moment Accumulator, Exponential Moving Average, Bias Correction, Per-Coordinate Learning Rate

Adagrad

AdaGrad (Duchi, Hazan, Singer, 2011) adapts the learning rate to each individual parameter based on the history of past squared gradients: 1. Accumulate squared gradients: s_t = s_{t-1} + g_t odot g_t 2. Parameter update: theta_{t+1} = theta_t - (eta / sqrt(s_t + epsilon)) odot g_t Merit: Automatically takes large steps for infrequent/sparse features and small steps for frequent features. Fatal Flaw: Since g_t^2 >= 0, s_t increases monotonically with every training step. Consequently, the effective learning rate eta / sqrt(s_t) strictly decays to zero, prematurely terminating learning before the network reaches the optimum!

Rmsprop

RMSProp (Geoff Hinton, Coursera Lecture 6e, 2012) fixes AdaGrad's premature freezing by replacing the monotonic sum with an Exponentially Weighted Moving Average (EWMA): 1. Update discounted second moment: s_t = beta_2 * s_{t-1} + (1 - beta_2) * (g_t odot g_t) (typically beta_2 = 0.9 or 0.99) 2. Parameter update: theta_{t+1} = theta_t - (eta / sqrt(s_t + epsilon)) odot g_t This allows RMSProp to discard stale gradient history and adapt dynamically to recent curvature.

Adam

Adam (Kingma & Ba, ICLR 2015) combines the advantages of Momentum (first moment estimation) and RMSProp (second moment estimation): 1. First moment (mean of gradient): m_t = beta_1 * m_{t-1} + (1 - beta_1) * g_t (default beta_1 = 0.9) 2. Second moment (uncentered variance): v_t = beta_2 * v_{t-1} + (1 - beta_2) * (g_t odot g_t) (default beta_2 = 0.999) 3. Bias corrections (counteracting initialization at zero): m_hat_t = m_t / (1 - beta_1^t) v_hat_t = v_t / (1 - beta_2^t) 4. Parameter update: theta_{t+1} = theta_t - (eta / (sqrt(v_hat_t) + epsilon)) odot m_hat_t (default eta = 0.001, epsilon = 1e-8)

Exam Important Points

- KTU Exam Q13(a): Mathematical explanation of AdaGrad gradient decay and why it halts training.
- Comparison table: SGD vs AdaGrad vs RMSProp vs Adam.
- Full mathematical derivation of Adam's bias correction terms 1 / (1 - beta^t).
- Why Adam is the de facto default optimizer across modern deep learning architectures.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): AdaGrad vs RMSProp Two-Step Accumulator Comparison

Statement: A parameter w has initial value 1.0. At step 1 and step 2, the observed gradients are g1 = 4.0 and g2 = 4.0. The base learning rate is eta = 0.5 and epsilon = 1e-8. For RMSProp, use decay factor beta_2 = 0.9. Calculate: (a) AdaGrad's accumulator s_1, s_2 and effective learning rates, and (b) RMSProp's accumulator s_1, s_2 and effective learning rates.

Given: g1 = 4.0, g2 = 4.0, eta = 0.5, epsilon = 1e-8, RMSProp beta_2 = 0.9.

Governing Formula: AdaGrad: s_t = s_{t-1} + g_t^2; RMSProp: s_t = beta_2 * s_{t-1} + (1 - beta_2) * g_t^2; eta_eff = eta / sqrt(s_t + eps)

Step-by-Step Calculation:

Step 1: AdaGrad calculations: s_1 = 0 + 4.0^2 = 16.0 -> eta_eff_1 = 0.5 / sqrt(16.0) = 0.5 / 4.0 = 0.1250. s_2 = 16.0 + 4.0^2 = 16.0 + 16.0 = 32.0 -> eta_eff_2 = 0.5 / sqrt(32.0) = 0.5 / 5.6569 = 0.0884.

Step 2: RMSProp calculations: s_1 = (0.9 * 0) + (1 - 0.9) * 4.0^2 = 0.1 * 16.0 = 1.60. eta_eff_1 = 0.5 / sqrt(1.60) = 0.5 / 1.2649 = 0.3953. s_2 = (0.9 * 1.60) + (0.1 * 16.0) = 1.44 + 1.60 = 3.04. eta_eff_2 = 0.5 / sqrt(3.04) = 0.5 / 1.7436 = 0.2868.

[FINAL EVALUATED ANSWER: AdaGrad: s_2 = 32.0, eta_eff = 0.0884. RMSProp: s_2 = 3.04, eta_eff = 0.2868.]

Pedagogical Note: AdaGrad's accumulator grew rapidly to 32.0, suffocating the effective step size by ~82%. RMSProp maintained a balanced discounted accumulator (3.04), allowing continued agile learning.

Problem 2 (Advanced (KTU Exam Level)): Adam Bias Correction Derivation and Step Calculation

Statement: At step t = 1, an Adam optimizer observes gradient g_1 = 2.0. Hyperparameters are eta = 0.01, beta_1 = 0.9, beta_2 = 0.999, and epsilon = 1e-8. Initial moments are m_0 = 0.0, v_0 = 0.0. (a) Calculate the raw moments m_1 and v_1, (b) Calculate the bias-corrected moments m_hat_1 and v_hat_1, (c) Explain why bias correction is mandatory at early steps, and (d) Compute the parameter update step delta_w.

Given: t = 1, g_1 = 2.0, eta = 0.01, beta_1 = 0.9, beta_2 = 0.999, eps = 1e-8, m_0 = 0, v_0 = 0.

Governing Formula: m_1 = beta_1*m_0 + (1 - beta_1)*g_1; v_1 = beta_2*v_0 + (1 - beta_2)*g_1^2; m_hat = m_1 / (1 - beta_1^1); v_hat = v_1 / (1 - beta_2^1); delta_w = - (eta / (sqrt(v_hat) + eps)) * m_hat

Step-by-Step Calculation:

Step 1: Compute raw moments: m_1 = (0.9 * 0) + (1 - 0.9) * 2.0 = 0.1 * 2.0 = 0.20. v_1 = (0.999 * 0) + (1 - 0.999) * (2.0)^2 = 0.001 * 4.0 = 0.0040.

Step 2: Compute bias correction denominators: 1 - beta_1^1 = 1 - 0.9 = 0.10. 1 - beta_2^1 = 1 - 0.999 = 0.0010.

Step 3: Compute bias-corrected moments: m_hat_1 = 0.20 / 0.10 = 2.00. v_hat_1 = 0.0040 / 0.0010 = 4.00.

Step 4: Compute parameter update step: delta_w = - (0.01 / (sqrt(4.00) + 1e-8)) * 2.00 = - (0.01 / 2.0) * 2.00 = - 0.0100.

[FINAL EVALUATED ANSWER: Raw: m_1=0.20, v_1=0.0040. Corrected: m_hat_1=2.00, v_hat_1=4.00. Step delta_w = -0.0100]

Pedagogical Note: Without bias correction, v_1 would be 0.0040, giving sqrt(v_1) = 0.0632 and resulting in a wildly distorted, inflated initial step. Bias correction normalizes both moments to reflect the true expected gradient scale.

Real-World Industry Case (Autonomous Robotics & Edge Spatial Computing): Autonomous Drone LiDAR Simultaneous Localization and Mapping (SLAM)

Pipeline Workflow: Drone flies through unmapped indoor warehouse -> LiDAR point cloud features are sparse (certain spatial bins trigger rarely while floor points trigger constantly) -> Adam optimizer adapts individual coordinate step sizes dynamically.

Industrial Impact & Benefit: Enables rapid convergence of 3D spatial pose graph estimation in sub-second real-time constraints.

Key Oral Viva Voce Checkpoints

Q1: What core problem in vanilla GD did AdaGrad aim to solve?

Ans: The reliance on a single global learning rate for all parameters, regardless of how frequently or sparsely individual features occurred.

Q2: What is the fatal flaw of the AdaGrad optimizer?

Ans: The monotonic accumulation of squared gradients in the denominator causes the effective learning rate to decay to near zero, prematurely halting training.

Q3: How does RMSProp overcome AdaGrad's learning rate decay problem?

Ans: By using an exponentially weighted moving average of squared gradients instead of an unweighted cumulative sum.

Q4: What two algorithms are combined to create Adam?

Ans: Momentum (first moment of gradients) and RMSProp (second moment of gradients).

Q5: Why are bias corrections needed in the Adam optimizer?

Ans: Because the first and second moment vectors are initialized to zero, biasing their estimates towards zero during the initial training steps; dividing by (1 - beta^t) removes this bias.

LECTURE #13 · MODULE 2 (Topic 2.6)

Date: 24-Sep-2025
Period: Period 4

Title: Regularization Techniques: L1, L2, Early Stopping, and Dataset Augmentation

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Loss Formulation, Gradient Updates, Optimization Overview, Day 9 concepts
Key Terminology: L1 Regularization (Lasso), L2 Regularization (Weight Decay / Ridge), Sparsity, Weight Shrinkage, Early Stopping, Patience, Dataset Augmentation

L1 And L2 Regularization

Regularized Objective Function: J_tilde(w) = J(w) + alpha * Omega(w) 1. L2 Regularization (Weight Decay / Ridge): Omega(w) = (1/2) * ||w||_2^2 = (1/2) * sum_i w_i^2 Gradient: grad J_tilde = grad J + alpha * w Update Step: w_{t+1} = w_t - eta * (grad J + alpha * w_t) = (1 - eta * alpha) * w_t - eta * grad J Effect: Multiplies the weight vector by a shrinkage factor (1 - eta * alpha) < 1 at each step, preventing weights from growing excessively large. 2. L1 Regularization (Lasso): Omega(w) = ||w||_1 = sum_i |w_i| Gradient: grad J_tilde = grad J + alpha * sign(w) Update Step: w_{t+1} = w_t - eta * grad J - eta * alpha * sign(w_t) Geometric Sparsity: The L1 constraint forms a diamond-shaped polytope with sharp corners on the coordinate axes. The elliptical loss contours intersect these corners, driving many weights to be exactly zero (feature selection).

Early Stopping

Early Stopping is the most widely used regularization technique in deep learning. Protocol: 1. At the end of each epoch, evaluate loss on a hold-out validation set. 2. If validation loss improves, save model parameters theta* = theta. 3. If validation loss does not improve for 'patience' P consecutive epochs, terminate training and restore theta*. Mathematical Equivalence: Bishop (1995) proved that on quadratic loss surfaces, early stopping after tau iterations is mathematically equivalent to L2 regularization with weight decay parameter alpha approx 1 / (eta * tau).

Dataset Augmentation

The most reliable method to improve generalization is to train on more data. When additional data is scarce, Dataset Augmentation generates realistic synthetic transformations: - Computer Vision: Random horizontal flips, random cropping, rotations (+-15 deg), color jitter, affine shearing, CutMix, MixUp. - Speech: Time stretching, pitch shifting, background noise injection (SpecAugment). - Text: Synonym replacement, back-translation (English -> German -> English).

Exam Important Points

- Derivation of weight shrinkage equation w <- (1 - eta*alpha)*w for L2 regularization.
- Geometric explanation with sketches showing why L1 produces sparsity while L2 shrinks weights uniformly.
- Early stopping protocol, patience parameter, and its equivalence to L2 regularization.
- List of data augmentation techniques for image classification.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): L2 Weight Decay Shrinkage Step Calculation

Statement: A neural network weight currently has value w_t = 0.80. The unregularized loss gradient with respect to this weight is dJ/dw = 0.10. The learning rate is eta = 0.05 and the L2 regularization coefficient is alpha = 0.20. Calculate: (a) The effective shrinkage factor (1 - eta * alpha), (b) The weight value after decay but before gradient application, and (c) The final updated weight w_{t+1}.

Given: w_t = 0.80, dJ/dw = 0.10, eta = 0.05, alpha = 0.20

Governing Formula: Shrinkage factor = 1 - eta * alpha; w_decayed = (1 - eta * alpha) * w_t; w_{t+1} = w_decayed - eta * (dJ/dw)

Step-by-Step Calculation:

Step 1: Compute shrinkage factor: factor = 1.0 - (0.05 * 0.20) = 1.0 - 0.010 = 0.990.

Step 2: Apply weight decay: w_decayed = 0.990 * 0.80 = 0.7920.

Step 3: Apply gradient descent step: grad_step = 0.05 * 0.10 = 0.0050. w_{t+1} = 0.7920 - 0.0050 = 0.7870.

[FINAL EVALUATED ANSWER: Shrinkage factor = 0.990, Decayed weight = 0.7920, Final weight w_{t+1} = 0.7870]

Pedagogical Note: L2 regularization systematically eroded the weight by 1.0% (from 0.80 to 0.792) prior to applying the empirical gradient descent update.

Problem 2 (Advanced (KTU Exam Level)): Optimal Weight Reduction under L2 Regularization on Quadratic Loss

Statement: Consider a scalar quadratic loss function J(w) = 0.5 * H * (w - w*)^2 where Hessian curvature H = 4.0 and unregularized optimum w* = 3.0. An L2 penalty term 0.5 * alpha * w^2 is added with alpha = 1.0. (a) Derive the expression for the regularized objective J_tilde(w), (b) Find the exact regularized optimal weight w_reg by setting dJ_tilde/dw = 0, and (c) Show that w_reg = (H / (H + alpha)) * w* and compute the percentage shrinkage.

Given: H = 4.0, w* = 3.0, alpha = 1.0, J_tilde(w) = 0.5 * H * (w - w*)^2 + 0.5 * alpha * w^2.

Governing Formula: dJ_tilde/dw = H*(w - w*) + alpha*w = 0 => (H + alpha)*w = H*w* => w_reg = (H / (H + alpha)) * w*

Step-by-Step Calculation:

Step 1: Differentiate regularized objective: dJ_tilde/dw = 4.0 * (w - 3.0) + 1.0 * w = 4.0*w - 12.0 + 1.0*w = 5.0*w - 12.0.

Step 2: Set derivative to zero to solve for optimum: 5.0 * w_reg - 12.0 = 0 => w_reg = 12.0 / 5.0 = 2.40.

Step 3: Verify theoretical ratio: Ratio = H / (H + alpha) = 4.0 / (4.0 + 1.0) = 4.0 / 5.0 = 0.80. w_reg = 0.80 * 3.0 = 2.40.

Step 4: Calculate shrinkage percentage: Shrinkage = ((3.0 - 2.40) / 3.0) * 100 = (0.60 / 3.0) * 100 = 20.0%.

[FINAL EVALUATED ANSWER: Regularized optimum w_reg = 2.40 (Exact 20% shrinkage from unregularized value 3.0)]

Pedagogical Note: The L2 penalty pulls the parameter toward the origin; directions with higher curvature H resist shrinkage more strongly than flatter directions with small H.

Real-World Industry Case (Healthcare Diagnostics & Radiomics): Medical Imaging CT Scan Lung Nodule Detection

Pipeline Workflow: Patient 3D CT pulmonary scans -> Extremely small training dataset (only 500 patient scans available) -> Massive 10x dataset augmentation (random 3D rotations, intensity scaling, elastic deformation) combined with L2 weight decay.

Industrial Impact & Benefit: Prevents catastrophic overfitting to patient scanner artifacts, boosting lung cancer early detection sensitivity to 96.4%.

Key Oral Viva Voce Checkpoints

Q1: Why does L1 regularization lead to sparse weight matrices while L2 does not?

Ans: L1 has a constant gradient magnitude alpha*sign(w) right down to the origin and its constraint surface has sharp corners on the axes where elliptical loss contours typically intersect; L2 gradient alpha*w shrinks proportionally to w, pulling weights close to zero without making them exactly zero.

Q2: What is the update rule for weight decay in L2 regularization?

Ans: w_{t+1} = (1 - eta * alpha) * w_t - eta * grad J(w_t).

Q3: What is the physical meaning of the 'patience' parameter in Early Stopping?

Ans: The number of consecutive epochs to wait for validation loss improvement before terminating training.

Q4: Why does Early Stopping prevent overfitting?

Ans: It stops optimization before the model begins memorizing idiosyncratic training set noise, constraining the parameter vector close to its initial state.

Q5: Can dataset augmentation be applied to tabular data?

Ans: Yes, techniques like SMOTE (Synthetic Minority Over-sampling Technique) or adding small Gaussian jitter to continuous features perform tabular augmentation.

LECTURE #14 · MODULE 2 (Topic 2.7)

Date: 26-Sep-2025
Period: Period 1

Title: Parameter Sharing, Parameter Tying, and Injecting Noise at Input

CO: CO2 Bloom's: Understand / Apply

FACULTY SYLLABUS NOTE: Official KTU 2.7 & 2.8 duplication flagged. This lecture covers: Parameter Sharing, Parameter Tying, and Injecting Noise at Input.

Prerequisites: Convolution Operation intuition, Autoencoder concepts, Regularization, Day 13 concepts
Key Terminology: Parameter Sharing, Parameter Tying, Tied Weights, Input Noise Injection, Tikhonov Regularization, Denoising Objective

Parameter Sharing And Tying

1. Parameter Sharing: Instead of assigning unique independent weights to every connection, parameter sharing forces multiple components to share the exact same parameter value. - Primary Example: Convolutional Neural Networks (CNNs). A convolutional filter (e.g. 3x3 kernel) is shared across all spatial locations of the input image. - Benefit: Slashes parameter count from millions to hundreds, enforces translation equivariance, and drastically lowers sample complexity. 2. Parameter Tying: Parameter tying imposes hard linear constraints between different weight matrices in a network. - Classic Example: Symmetric Autoencoders where decoder weights W_d are tied to the transpose of encoder weights W_e: W_d = W_e^T - Benefit: Halves the number of parameters in the autoencoder and acts as a strong regularizer preventing the latent representation from learning identity trivialities.

Injecting Noise At Input

Adding random noise epsilon ~ N(0, sigma^2 * I) to the input vector x during training: x_tilde = x + epsilon Mathematical Analysis (Bishop, 1995): For a linear or smooth model f(x) trained with squared error loss, minimizing the expected loss over noisy inputs: E_epsilon [0.5 * (f(x + epsilon) - y)^2] is mathematically equivalent to minimizing the uncorrupted loss PLUS an L2 regularization penalty proportional to the Frobenius norm of the model's Jacobian: L_regularized = L_clean + (1/2) * sigma^2 * || grad_x f(x) ||_2^2 This penalizes rapid changes in the model's prediction with respect to input perturbations, forcing the network to learn a smooth, robust decision boundary.

Exam Important Points

- Definition and architectural benefits of Parameter Sharing in CNNs and RNNs.
- Formulation of Tied Weights in Autoencoders: W_decoder = W_encoder^T.
- Mathematical proof that injecting input Gaussian noise is equivalent to Tikhonov gradient penalty regularization.
- How input noise improves model robustness against adversarial attacks.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Parameter Reduction Calculation in Tied Autoencoder

Statement: An autoencoder maps an input vector of dimension n_in = 784 into a compressed latent bottleneck of dimension n_h = 64, and then reconstructs the 784-dimensional output. (a) Calculate the total trainable weights and biases if encoder and decoder weights are UNTIED, (b) Calculate the total parameters if weights are TIED (W_decoder = W_encoder^T), and (c) Determine the exact percentage savings in parameter storage.

Given: n_in = 784, n_h = 64, n_out = 784.

Governing Formula: Untied Params = 2 * (n_in * n_h) + n_h + n_in; Tied Params = (n_in * n_h) + n_h + n_in

Step-by-Step Calculation:

Step 1: Compute Untied parameter count: Encoder weights = 64 * 784 = 50,176. Encoder biases = 64. Decoder weights = 784 * 64 = 50,176. Decoder biases = 784. Total Untied Parameters = 50,176 + 64 + 50,176 + 784 = 101,200.

Step 2: Compute Tied parameter count: Shared weight matrix W = 64 * 784 = 50,176. Encoder biases = 64. Decoder biases = 784. Total Tied Parameters = 50,176 + 64 + 784 = 51,024.

Step 3: Compute parameter reduction: Difference = 101,200 - 51,024 = 50,176 parameters. Percentage Savings = (50,176 / 101,200) * 100 = 49.58%.

[FINAL EVALUATED ANSWER: Untied = 101,200 params. Tied = 51,024 params (49.58% parameter reduction).]

Pedagogical Note: Tying weights virtually cuts the network weight parameters in half while enforcing symmetric encoder-decoder representations.

Problem 2 (Advanced (KTU Exam Level)): Mathematical Equivalence of Input Noise to Tikhonov Regularization

Statement: Let a linear model be f(x) = w^T * x with scalar squared error loss L = 0.5 * (f(x) - y)^2. Add isotropic Gaussian noise epsilon with mean 0 and variance sigma^2 * I to input x, giving perturbed input x_tilde = x + epsilon. (a) Expand the expected loss E_epsilon [0.5 * (w^T * (x + epsilon) - y)^2], and (b) Prove algebraically that the expected noisy loss equals the clean loss plus an exact L2 penalty 0.5 * sigma^2 * ||w||_2^2.

Given: f(x) = w^T * x, x_tilde = x + epsilon, E[epsilon] = 0, E[epsilon * epsilon^T] = sigma^2 * I.

Governing Formula: w^T * (x + epsilon) - y = (w^T*x - y) + w^T*epsilon; Expand square and take expectation.

Step-by-Step Calculation:

Step 1: Express error term: e(epsilon) = (w^T * x - y) + w^T * epsilon. Let e_clean = w^T * x - y.

Step 2: Square the error term: e(epsilon)^2 = e_clean^2 + 2 * e_clean * (w^T * epsilon) + (w^T * epsilon)^2.

Step 3: Take mathematical expectation with respect to noise distribution epsilon: E[e(epsilon)^2] = E[e_clean^2] + 2 * e_clean * w^T * E[epsilon] + E[(w^T * epsilon)^2].

Step 4: Since noise is zero-mean, E[epsilon] = 0, so the middle cross-term vanishes: 2 * e_clean * w^T * 0 = 0.

Step 5: Evaluate the quadratic noise term: (w^T * epsilon)^2 = (w^T * epsilon) * (epsilon^T * w) = w^T * (epsilon * epsilon^T) * w. E[w^T * (epsilon * epsilon^T) * w] = w^T * E[epsilon * epsilon^T] * w = w^T * (sigma^2 * I) * w = sigma^2 * ||w||_2^2.

Step 6: Combine results with the 0.5 loss factor: E[L(x_tilde)] = 0.5 * e_clean^2 + 0.5 * sigma^2 * ||w||_2^2 = L_clean + 0.5 * sigma^2 * ||w||_2^2.

[FINAL EVALUATED ANSWER: E[L(x_tilde)] = L_clean + 0.5 * sigma^2 * ||w||_2^2 (Exact L2 Regularization Penalty!)]

Pedagogical Note: This rigorous proof establishes that adding input Gaussian noise with variance sigma^2 is mathematically identical to applying explicit L2 weight decay with regularization coefficient alpha = sigma^2.

Real-World Industry Case (Audio Signal Processing & In-Cabin Infotainment): Noise-Resistant Automatic Speech Recognition in High-Speed Rail

Pipeline Workflow: High-speed rail passenger speaks voice commands into cabin microphone -> Severe acoustic rumble (wind shear, steel track vibration) -> Network trained with Gaussian noise injection at input audio spectrogram frames.

Industrial Impact & Benefit: Maintains 94% command recognition accuracy without acoustic degradation or word drops under 300 km/h train speeds.

Key Oral Viva Voce Checkpoints

Q1: What is Parameter Sharing in neural networks?

Ans: Using the same set of weights across multiple computational units or time steps, rather than learning independent weights for each connection.

Q2: Give two prominent examples of parameter sharing in modern architectures.

Ans: Convolutional Neural Networks (sharing filter kernels across image space) and Recurrent Neural Networks (sharing transition matrices across time steps).

Q3: What is Parameter Tying in Autoencoders?

Ans: Constraining the decoder weight matrix to be the exact transpose of the encoder weight matrix: W_decoder = W_encoder^T.

Q4: How does parameter tying benefit an autoencoder?

Ans: It halves the model parameter count, prevents overfitting, and prevents the hidden representation from learning a trivial identity mapping.

Q5: What happens mathematically when Gaussian noise is added to model inputs during training?

Ans: It acts as an implicit regularizer mathematically equivalent to an L2 weight penalty proportional to the noise variance.

LECTURE #15 · MODULE 2 (Topic 2.8)

Date: 29-Sep-2025
Period: Period 2

Title: Ensemble Methods, Model Averaging, Bagging and Boosting in Deep Learning

CO: CO2 Bloom's: Understand / Apply

FACULTY SYLLABUS NOTE: Official KTU 2.7 & 2.8 duplication flagged. This lecture covers: Ensemble Methods, Model Averaging, Bagging and Boosting in Deep Learning.

Prerequisites: Bias-Variance Decomposition, Generalization Theory, Day 14 concepts
Key Terminology: Ensemble Learning, Model Averaging, Bootstrap Aggregation (Bagging), Boosting, Snapshot Ensembles, Variance Reduction, Syllabus Duplication Verification

Faculty Syllabus Flag

FACULTY NOTICE: In the approved KTU CST414 teaching plan (Page 12 of DEEP LEARNING.pdf), topics 2.7 and 2.8 carry the identical title: 'Parameter sharing and tying, Injecting noise at input, Ensemble methods'. To ensure complete academic coverage without redundant lecturing, Day 14 covered Parameter Sharing, Tying, and Input Noise, while Day 15 is dedicated thoroughly to Ensemble Methods, Model Averaging, Bagging, and Snapshot Ensembles.

Model Averaging Mathematics

Ensemble learning combines the predictions of k distinct models {f_1(x), f_2(x), ..., f_k(x)}: f_ens(x) = (1 / k) * sum_{i=1}^k f_i(x) Variance Reduction Theorem: Suppose each model f_i has error epsilon_i with variance E[epsilon_i^2] = v and covariance E[epsilon_i * epsilon_j] = c between different models. The expected squared error of the ensemble predictor is: E[((1/k) * sum_i epsilon_i)^2] = (1 / k^2) * sum_i sum_j E[epsilon_i * epsilon_j] = (1 / k^2) * [k * v + k * (k - 1) * c] = (1 / k) * v + ((k - 1) / k) * c Two Extreme Cases: 1. Perfectly Correlated Models (c = v): Ensemble error = (1/k)*v + ((k-1)/k)*v = v. No gain. 2. Perfectly Uncorrelated Errors (c = 0): Ensemble error = v / k. Error decreases linearly with ensemble size k! In deep learning, even if models share the same architecture, different random weight initializations and mini-batch shuffle orders lead to partially uncorrelated errors (c < v), guaranteeing that the ensemble outperforms any single constituent model.

Deep Ensemble Techniques

1. Bagging (Bootstrap Aggregation): Train k identical architectures on k bootstrap resamples of the dataset. 2. Multi-Architecture Ensembling: Average predictions of diverse models (e.g. ResNet + DenseNet + Vision Transformer). 3. Snapshot Ensembles (Huang et al., 2017): Use a cyclical learning rate schedule to converge to multiple local minima during a single training run, saving model snapshots at the base of each cycle without k-fold training cost.

Exam Important Points

- Mathematical derivation of ensemble error: E[e^2] = (1/k)*v + ((k-1)/k)*c.
- Why ensemble error strictly improves whenever model error correlation c < v.
- Comparison of Bagging vs Boosting in deep learning contexts.
- Snapshot ensembles: achieving ensemble benefits in a single training run.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Theoretical Ensemble Error Variance Calculation

Statement: An ensemble is composed of k = 5 independently trained deep neural networks. Each individual network has an expected squared error variance of v = 0.20. The average pairwise covariance between the errors of different models is c = 0.05. (a) Calculate the expected squared error of the ensemble predictor, and (b) Calculate the percentage reduction in error achieved compared to using any single individual model.

Given: k = 5 models, individual variance v = 0.20, covariance c = 0.05.

Governing Formula: E_ens = (1/k) * v + ((k - 1) / k) * c; Reduction = (v - E_ens) / v * 100

Step-by-Step Calculation:

Step 1: Compute first term (variance contribution): term1 = v / k = 0.20 / 5 = 0.040.

Step 2: Compute second term (covariance contribution): term2 = ((5 - 1) / 5) * 0.05 = (4 / 5) * 0.05 = 0.80 * 0.05 = 0.040.

Step 3: Calculate total ensemble expected error: E_ens = term1 + term2 = 0.040 + 0.040 = 0.080.

Step 4: Calculate percentage reduction: Reduction = ((0.20 - 0.080) / 0.20) * 100 = (0.120 / 0.20) * 100 = 60.0%.

[FINAL EVALUATED ANSWER: Ensemble Error = 0.080 (60% error reduction compared to individual model error 0.20)]

Pedagogical Note: Because the model errors were only weakly correlated (c = 0.05 vs v = 0.20), model averaging slashed the total error variance by 60%.

Problem 2 (Advanced (KTU Exam Level)): Ensemble Probability Voting vs Majority Hard Voting

Statement: Three classifiers (M1, M2, M3) output predicted probabilities for a binary positive class as P1 = 0.45, P2 = 0.48, and P3 = 0.95 for a true positive instance (y = 1). Using a classification threshold of 0.50: (a) Determine the individual binary classifications of M1, M2, and M3, (b) Determine the ensemble prediction under Majority Hard Voting, (c) Calculate the Soft Average ensemble probability P_soft and determine its prediction, and (d) Contrast the robustness of soft vs hard voting.

Given: P1 = 0.45, P2 = 0.48, P3 = 0.95. True class y = 1. Threshold = 0.50.

Governing Formula: Hard Vote: argmax(count(pred == 1)); Soft Vote: P_soft = (1/3) * (P1 + P2 + P3)

Step-by-Step Calculation:

Step 1: Individual binary decisions: M1: P1 = 0.45 < 0.50 -> Class 0 (Incorrect) M2: P2 = 0.48 < 0.50 -> Class 0 (Incorrect) M3: P3 = 0.95 >= 0.50 -> Class 1 (Correct).

Step 2: Majority Hard Voting: Votes for Class 0 = 2 (M1, M2). Votes for Class 1 = 1 (M3). Hard Voting Ensemble Prediction = Class 0 (WRONG!).

Step 3: Soft Probability Voting: P_soft = (0.45 + 0.48 + 0.95) / 3 = 1.88 / 3 = 0.6267. Since P_soft = 0.6267 >= 0.50, Soft Voting Ensemble Prediction = Class 1 (CORRECT!).

[FINAL EVALUATED ANSWER: Hard Vote = Class 0 (Fails); Soft Vote = Class 1 with 62.67% confidence (Succeeds)]

Pedagogical Note: Soft probability voting takes into account confidence calibration: M3 was extremely confident (95%) while M1 and M2 were barely uncertain (45% and 48%). Soft voting correctly allowed M3's strong signal to overturn the marginal majority.

Real-World Industry Case (Competitive Machine Learning & Financial Risk Modeling): Kaggle Grandmaster Competition Winning Pipeline

Pipeline Workflow: Tabular credit loan dataset -> Team trains 10 diverse models (LightGBM, XGBoost, CatBoost, 5-layer MLP, ResNet-Tabular) -> Out-of-fold probability predictions combined using weighted soft voting ensemble.

Industrial Impact & Benefit: Ensemble consistently outscores any individual model by 0.015-0.030 AUC, making the decisive difference between 1st place gold and 50th place.

Key Oral Viva Voce Checkpoints

Q1: What is the primary mathematical mechanism through which ensembling reduces test error?

Ans: Variance reduction: averaging uncorrelated error terms dampens overall prediction variance without increasing bias.

Q2: Why is ensembling particularly effective for deep neural networks?

Ans: Because deep networks have highly non-convex loss surfaces; different initializations and mini-batch sequences cause models to converge to distinct local minima that make different errors.

Q3: What is the difference between Bagging and Boosting?

Ans: Bagging trains independent models in parallel on bootstrap resamples to reduce variance; Boosting trains models sequentially, focusing each new model on earlier errors to reduce bias.

Q4: What is the key disadvantage of deploying an ensemble of 10 deep neural networks to production?

Ans: Inference latency and memory costs are multiplied by 10, which may be unacceptable for real-time edge applications.

Q5: What is a Snapshot Ensemble?

Ans: A technique using cyclical learning rate schedules to visit multiple local minima in a single training run, saving model weights at each cycle trough to form an ensemble for free.

LECTURE #16 · MODULE 2 (Topic 2.9)

Date: 03-Oct-2025
Period: Period 1

Title: Dropout and Parameter Initialization

CO: CO2 Bloom's: Understand / Apply

Prerequisites: Activation Functions, Backpropagation, Ensemble Theory, Day 15 concepts
Key Terminology: Dropout, Bernoulli Mask, Inverted Dropout, Co-Adaptation, Xavier/Glorot Initialization, He/Kaiming Initialization, Fan-In / Fan-Out, Variance Preservation

Dropout Formulation

Dropout (Srivastava et al., JMLR 2014) is a computationally efficient regularization technique that prevents feature co-adaptation. 1. Training Phase: For each training example and each training step, each hidden unit is randomly retained with probability p (or dropped with drop rate q = 1 - p): r^(l) ~ Bernoulli(p) h_tilde^(l) = r^(l) odot h^(l) 2. Inverted Dropout (Standard Modern Implementation): To ensure that the expected output during testing matches training without requiring post-hoc test-time scaling, activations are scaled by 1/p during training: h_tilde^(l) = (r^(l) odot h^(l)) / p 3. Test Phase: At test time, Dropout is deactivated completely: h^(l) is evaluated deterministically without scaling. Ensemble Interpretation: In a network with N units, Dropout samples from an ensemble of 2^N distinct 'thinned' sub-networks that all share the exact same parameter weights! At test time, using the full unmasked network computes the geometric mean of the sub-networks' predictive distributions.

Parameter Initialization

Why not initialize weights to zero? Symmetry problem: all neurons compute identical forward activations and receive identical backpropagated gradients. Why not large random weights? Activations explode or saturate (vanishing gradients in sigmoid/tanh). 1. Xavier / Glorot Initialization (Glorot & Bengio, 2010): Designed for Tanh and Sigmoid activations. Goal: maintain constant variance of activations during forward pass and gradients during backward pass: Var(W) = 2 / (fan_in + fan_out) Uniform distribution: W ~ U(- sqrt(6 / (fan_in + fan_out)), + sqrt(6 / (fan_in + fan_out))) Normal distribution: W ~ N(0, 2 / (fan_in + fan_out)) 2. He / Kaiming Initialization (He et al., ICCV 2015): Designed specifically for ReLU activations (which zero out half the distribution): Var(W) = 2 / fan_in Uniform distribution: W ~ U(- sqrt(6 / fan_in), + sqrt(6 / fan_in)) Normal distribution: W ~ N(0, sqrt(2 / fan_in))

Exam Important Points

- KTU Exam Q14(b): Effect of hidden unit capacity combined with dropout on bias and variance.
- Full algorithmic steps of Inverted Dropout vs Classical Dropout.
- Ensemble of 2^N sub-networks interpretation of Dropout.
- Formulas and derivations for Xavier/Glorot vs He/Kaiming weight initialization.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Inverted Dropout Expected Value Verification

Statement: A hidden layer produces activation vector h = [2.0, 4.0, 6.0, 8.0]^T. During training, Inverted Dropout with retention probability p = 0.5 is applied. The generated Bernoulli mask is r = [1, 0, 1, 0]^T. (a) Calculate the masked activations under classical dropout and the required test-time scaling, (b) Calculate the masked activations under Inverted Dropout, and (c) Show that the expected sum of activations is preserved under Inverted Dropout.

Given: h = [2.0, 4.0, 6.0, 8.0]^T, p = 0.5, r = [1, 0, 1, 0]^T.

Governing Formula: Classical Train: h_tilde = r * h; Classical Test: h_test = p * h; Inverted Train: h_tilde = (r * h) / p

Step-by-Step Calculation:

Step 1: Compute classical dropout activations: h_drop = [2.0*1, 4.0*0, 6.0*1, 8.0*0]^T = [2.0, 0.0, 6.0, 0.0]^T. At test time, all weights or activations must be multiplied by p = 0.5.

Step 2: Compute Inverted Dropout activations: h_inv = h_drop / p = [2.0/0.5, 0.0/0.5, 6.0/0.5, 0.0/0.5]^T = [4.0, 0.0, 12.0, 0.0]^T.

Step 3: Compare expected sums: Original sum = 2.0 + 4.0 + 6.0 + 8.0 = 20.0. Inverted Dropout sum for this mask = 4.0 + 0.0 + 12.0 + 0.0 = 16.0. Expected sum across all masks = E[r_i] / p * sum(h_i) = (0.5 / 0.5) * 20.0 = 20.0.

[FINAL EVALUATED ANSWER: Classical: [2.0, 0.0, 6.0, 0.0]^T (requires test scaling). Inverted: [4.0, 0.0, 12.0, 0.0]^T (no test scaling required).]

Pedagogical Note: Inverted dropout scales surviving activations by 1/p (here, doubles them to 4.0 and 12.0) during training, perfectly matching the expected activation magnitude at test time.

Problem 2 (Advanced (KTU Exam Level)): Xavier vs He Initialization Standard Deviation Calculation

Statement: A dense layer connects a preceding layer with fan_in = 512 neurons to a subsequent layer with fan_out = 128 neurons. (a) Calculate the standard deviation sigma_xavier for Xavier/Glorot normal initialization, (b) Calculate the uniform bounds [-a, +a] for Xavier uniform initialization, (c) Calculate the standard deviation sigma_he for He/Kaiming normal initialization, and (d) Explain why sigma_he is larger than sigma_xavier.

Given: fan_in = 512, fan_out = 128.

Governing Formula: sigma_xavier = sqrt(2 / (fan_in + fan_out)); a_xavier = sqrt(6 / (fan_in + fan_out)); sigma_he = sqrt(2 / fan_in)

Step-by-Step Calculation:

Step 1: Xavier Normal standard deviation: Var_xavier = 2.0 / (512 + 128) = 2.0 / 640 = 0.003125. sigma_xavier = sqrt(0.003125) = 0.05590.

Step 2: Xavier Uniform bounds: a_xavier = sqrt(6.0 / (512 + 128)) = sqrt(6.0 / 640) = sqrt(0.009375) = 0.09682. Bounds: [-0.09682, +0.09682].

Step 3: He Normal standard deviation: Var_he = 2.0 / fan_in = 2.0 / 512 = 0.0039062S. sigma_he = sqrt(0.0039062S) = 0.06250.

Step 4: Comparison explanation: sigma_he (0.0625) > sigma_xavier (0.0559) because ReLU sets approximately half of all activations to zero (mean activation halved). To compensate for the 50% lost energy and maintain constant activation variance, He initialization injects a factor of 2 in the numerator.

[FINAL EVALUATED ANSWER: sigma_xavier = 0.0559, a_xavier = 0.0968, sigma_he = 0.0625]

Pedagogical Note: He initialization requires higher variance (0.0625 vs 0.0559) to prevent signal attenuation caused by ReLU clipping negative activations.

Real-World Industry Case (Computational Biology & Structural Bioinformatics): Protein Folding & Biomolecular Structure Prediction

Pipeline Workflow: Deep Transformer-based architecture processes amino acid sequences -> Severe risk of memorizing known crystallized proteins -> Monte Carlo Dropout (Gal & Ghahramani, 2016) enabled at inference time to generate epistemic uncertainty confidence intervals.

Industrial Impact & Benefit: Enables drug discovery scientists to flag uncertain protein loop predictions before committing to wet-lab synthesized assays.

Key Oral Viva Voce Checkpoints

Q1: What is the primary motivation behind the Dropout technique?

Ans: To prevent complex co-adaptation of hidden units, forcing each neuron to learn robust, generalizable features that do not rely on the presence of specific other neurons.

Q2: What is Inverted Dropout and why is it preferred over classical Dropout?

Ans: Inverted Dropout scales active neurons by 1/p during training, ensuring that test-time execution requires no scaling or architectural modification.

Q3: How does Dropout approximate an ensemble of models?

Ans: A network with N units samples from an ensemble of 2^N distinct thinned sub-networks that all share parameters, combining their predictions at test time.

Q4: What is the recommended retention probability p for hidden units vs input units?

Ans: Typically p = 0.5 for hidden layers, and p = 0.8 to 0.9 (or no dropout) for input layers.

Q5: Why is Xavier initialization unsuitable for deep ReLU networks?

Ans: Because ReLU sets all negative pre-activations to zero, halving the variance at each layer; Xavier initialization fails to account for this 50% loss, causing activation variance to decay exponentially to zero in deep networks.